TECNOLOGIE DEL WEB

2025/2026

Project Requirements

Traditional Development & LLM-Assisted Redevelopment

Prof. Alfonso Pierantonio
alfonso.pierantonio@univaq.it

1. Overview

The project consists in designing and implementing a Web application according to the methodology and the technologies illustrated during the course. The project is structured in two phases:

- **Phase 1 — Traditional Development:** the team designs and implements the application from scratch using the prescribed technology stack, following an iterative development process.
- **Phase 2 — LLM-Assisted Redevelopment:** the team redevelops the same application from scratch using one or more Large Language Models as development assistants. The LLM-assisted version must replicate all functionality of the Phase 1 application and may extend it with additional features or improvements.

Both phases contribute to the final evaluation. The dual-phase structure is designed to provide first-hand experience of the differences between traditional and AI-assisted software development, fostering critical awareness of the strengths and limitations of each approach.

2. Technology Stack

The application must be realized with a combination of the following technologies:

- PHP
- MySQL
- HTML / CSS
- JavaScript (vanilla, no jQuery)
- Templating (*template2.inc.php* distributed during the course)

> **Important:** it is not allowed to use frameworks such as Laravel. The aim of the course is presenting the foundational concepts of web application design. The use of frameworks would not permit to fully grasp such concepts as they operate at a higher level of abstraction.

Other template engines must be agreed with the lecturer in advance.

3. Project Size

The application must have at least 14 SQL tables. This number refers to the overall count of tables including relation normalizations (junction tables). The chosen application domain must be rich enough to naturally require this level of data modeling complexity.

4. Methodology

The application must be realized by adopting the following methodological pillars. Missing one of the below may correspond to a non-sufficient score for the project.

4.1 Separation of Logics

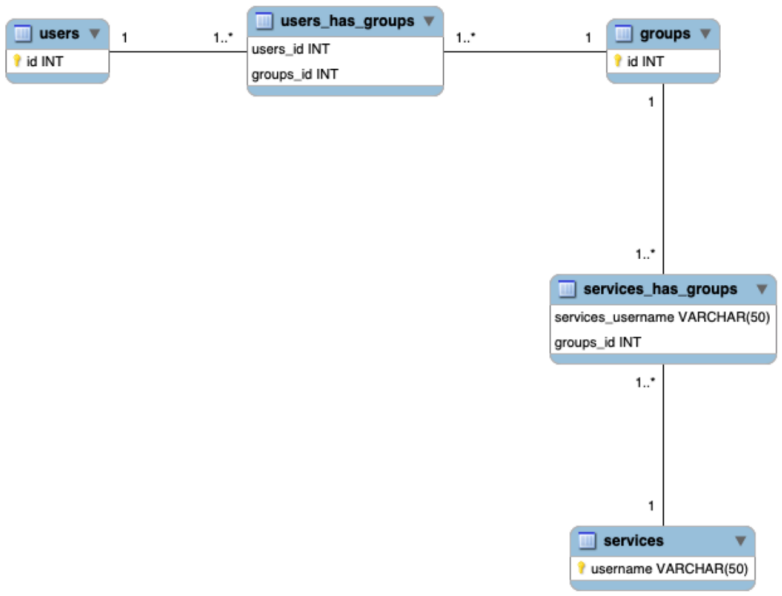
The separation of logics must be realized by using *template2.inc.php* illustrated and distributed during the course. This enforces a clear boundary between presentation, business logic, and data access.

4.2 Session Management

The application must use session management for realizing session-persistent abstractions such as e-commerce carts, authentication state, user preference management, and similar stateful behaviors.

4.3 Generic User Management

The application must implement an advanced user management system based on the classical model **users – groups – services**, as outlined in the following schema:



The above schema is simplified and only contains the relevant structural information (primary and foreign keys). The actual implementation will require additional attributes.

> **Important:** not fulfilling the users-groups-services requirement corresponds to a rejection of the project.

5. Development Process

5.1 Phase 1: Traditional Development

In Phase 1, the team must follow an iterative development process. The application must be segmented into functional slices, and for each slice the team must implement:

1. The relevant portion of the database schema
2. The corresponding CRUD operations in the administration backend
3. The corresponding frontend pages and interactions

Each iteration produces a working, testable increment of the application. This approach prevents the common pitfall of designing the entire database without implementing any functional pages, and reflects professional incremental development practices.

At the end of Phase 1, the application must be fully functional and satisfy all requirements described in this document.

5.2 Phase 2: LLM-Assisted Redevelopment

In Phase 2, the team redevelops the application from scratch using one or more Large Language Models (e.g., ChatGPT, Claude, Gemini, Copilot, or any other available LLM). The team is free to use a single LLM or a combination of different models.

Scope constraint: the LLM-assisted version must implement at least all the functionality present in the Phase 1 version. It may diverge only in a *conservative* direction, meaning it can improve or extend the application but must not reduce its scope. Possible improvements include:

- Cleaner, better-structured code
- More refined user interface
- More robust input validation and error handling
- Improved security practices
- Additional features that were not implemented in Phase 1

Process freedom: unlike Phase 1, the team is not required to follow the iterative slice-by-slice process. They are free to adopt any development workflow they find effective when working with the LLM.

Documentation requirement: the entire LLM-assisted development process must be thoroughly documented (see Section 8).

6. Graphic Layout

The team must choose two separate templates:

- **Frontend template:** must be appropriate for the application domain and effectively communicate the application's purpose and identity to end users. The template can be sourced from template repositories or existing websites.

- **Backend template:** must prioritize usability and productivity for administrators managing the application data. The backend may adopt a different visual language from the frontend.

The evaluation of the graphic layout will assess the appropriateness of the chosen templates for the proposed application, regardless of whether they were downloaded or created from scratch.

In Phase 2, the team may choose different templates if the LLM-assisted version benefits from an improved layout. Any change must be justified in the comparative report.

7. Administration Dashboard

The administration backend will be evaluated on three dimensions:

- **Coverage:** the application must provide all CRUD operations needed to manage every dynamic aspect of the application.
- **Usability:** the interface must be intuitive and provide a comfortable working environment.
- **Functionality:** the dashboard must be well-organized to maximize administrator productivity.

8. Deliverables

8.1 Phase 1 Deliverables

- Complete, functional application deployed online or installable on a laptop
- GitHub repository with individual commits from each team member
- Entity/Relationship diagram (printed or electronic)

8.2 Phase 2 Deliverables

- Complete, functional LLM-assisted application deployed online or installable on a laptop
- Separate GitHub repository (or clearly separated branch) with individual commits
- Full prompt log: complete record of all prompts submitted to the LLM(s)
- Conversation screenshots: visual documentation of LLM interactions, including the model's responses
- Development diary: a day-by-day journal documenting what was attempted, what worked, what failed, and what adjustments were made

8.3 Comparative Report

The team must submit a written comparative report analyzing the two development experiences. The report must follow the mandatory structure outlined below.

#	Section	Expected Content
1	**Executive Summary**	Overview of both phases and key findings (1 page max).
2	**Application Description**	Domain, main features, target users, and rationale for the chosen application.
3	**Development Process Comparison**	How did the workflow differ between the two phases? Time allocation, task decomposition, iteration patterns, team coordination.
4	**Code Quality Analysis**	Structural differences in the produced code: organization, readability, consistency, security practices, error handling. Include concrete examples.
5	**Feature Comparison**	Feature-by-feature comparison. What was improved, what was added, what remained equivalent. Use a comparison matrix if helpful.
6	**LLM Interaction Analysis**	Which LLM(s) were used and why. Effectiveness of different prompting strategies. Cases where the LLM excelled vs. where it required significant correction. Types of errors or hallucinations encountered.
7	**Effort and Productivity**	Estimated time spent on each phase. Perceived productivity differences. Where did the LLM save time? Where did it introduce overhead?
8	**Critical Reflection**	What did you learn about your own understanding of web development through the comparison? How did having built it manually first change your interaction with the LLM? Would the result have been different if you had used the LLM first?
9	**Conclusions**	Key takeaways on the role of LLMs in web development. Recommendations for when traditional vs. LLM-assisted development is more appropriate.

> **Important:** the comparative report is a critical component of the evaluation. Superficial or generic reports that do not demonstrate genuine reflection will negatively impact the final score. Make sure not to use LLM for generating the document.

9. Project Team

The project must be realized in teams of 2–3 students. If the team consists of 4 members, the project dimension must be increased by 25%. Teams of a single person are not allowed; exceptional cases must be discussed and agreed with the lecturer.

The project must be managed on GitHub. All commits must be performed with individual accounts — no shared team accounts are permitted. It must be possible to assess an equal contribution to the project by all team members across both phases.

10. Material for Examination

During the exam, both versions of the application must be usable and freely accessible by the lecturer:

- The applications can be deployed online
- The applications can be installed on a laptop

The following materials must be available (printed or electronically):

1. Entity/Relationship diagram for the database schema
2. Phase 1 application (deployed or installable)
3. Phase 2 application (deployed or installable)
4. Prompt log, conversation screenshots, and development diary
5. Comparative report following the mandatory structure

> **Important:** the lecturer might not evaluate projects that do not fulfill the above requirements.

11. Project Revisions

The teams can request a project review before the examination during student hours. Possible reviews relate to the following aspects:

- **Application concept:** send an email describing the kind of application and the domain (4–5 lines of text)
- **Graphic layout:** send an email with the URL of one or more templates, describing the application
- **Database schema:** by appointment only
- **General aspects:** by appointment only
- **LLM strategy:** send an email describing which LLM(s) you plan to use and your intended workflow for Phase 2

12. How to Contact the Lecturer

You can contact the lecturer via email at **alfonso.pierantonio@univaq.it** to request an appointment. You can also book an appointment directly from the lecturer's homepage:

https://disim.univaq.it/AlfonsoPierantonio